

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: J Satya Srikar

Roll No.: A24126552259

1. TITLE

Develop Dynamic Webpage Using Java Script

2. AIM

To design and develop a dynamic webpage utilizing JavaScript to capture user form submissions, interact with HTML DOM elements, obtain the user's geolocation coordinates via browser APIs, and fetch real-time weather and address data from third-party Web APIs using asynchronous requests.

3. OBJECTIVE

The objectives of this experiment are:

- To interact with the HTML Document Object Model (DOM) dynamically to retrieve and update page elements.
 - To capture user input events and block page refresh using standard event listeners.
 - To utilize the HTML5 Geolocation API (navigator.geolocation) to fetch GPS coordinates.
 - To implement asynchronous data fetching from REST Web APIs using the Fetch API and async/await.
 - To perform reverse geocoding via Nominatim API and fetch real-time weather parameters.
 - To dynamically modify page styling (background color) based on runtime data thresholds.
-

4. THEORY

A dynamic webpage uses scripting to change its content, styles, and behaviors dynamically in response to user actions or data queries. In this experiment, JavaScript acts as the scripting engine, interacting with the Document Object Model (DOM) to read inputs and write outputs without reloading the page.

Additionally, standard browser web APIs, such as the Geolocation API, and network utilities (Fetch API) are combined to gather coordinates and request real-time data from external APIs asynchronously.

Theory of DOM Manipulation & Asynchronous Web APIs:

- **DOM Event Handling:** Events are captured using *addEventListener*. Upon form submission, *e.preventDefault()* is executed to halt default page reloads, allowing JavaScript to capture values via *document.getElementById().value* and set headings using *innerText*.
 - **HTML5 Geolocation API:** Accessed via the global *navigator.geolocation* object. The *getCurrentPosition()* method queries browser permissions and returns latitude/longitude coordinates asynchronously.
 - **Asynchronous Network Queries:** Managed via the Fetch API (*fetch()* method). Functions are marked *async* to unlock promise handling. Inside, the execution is suspended using *await* until the HTTP promise settles, converting raw responses to JSON objects via *response.json()*.
 - **Nominatim Reverse Geocoding API:** Nominatim translates latitude/longitude parameters into human-readable locations. Address mappings yield city/town names which are dynamically written to the page header.
 - **OpenWeatherMap REST API:** Retrieves real-time temperature values. Absolute Kelvin temperatures are converted to Celsius ($K - 273.15$). If the temperature exceeds 30°C, the page background is styled red (*document.body.style.backgroundColor = 'red'*); otherwise, it is styled blue.
-

5. ALGORITHM / PROCEDURE

1. Establish project workspace folders: *Record/Experiment4-Week4/src* and *Docs*.
 2. Create *Dynamic.html* in the *src* directory.
 3. Assemble the HTML skeleton: define a city text input, submission form, location/temperature query buttons, and output placeholders.
 4. Add a script tag and bind a submit event listener to the form, blocking default refreshes and updating the selected city header.
 5. Bind a click event listener to the location button, calling the Geolocation API to fetch GPS coordinates.
 6. Implement async fetch queries targeting Nominatim API using longitude/latitude parameters to display the city name.
 7. Bind a click event listener to the temperature button, querying OpenWeatherMap API and displaying temperatures in Celsius.
 8. Configure conditional checks: alter page background color based on threshold value ($> 30^{\circ}\text{C}$ vs $\leq 30^{\circ}\text{C}$).
 9. Run page locally in Chrome, perform search and location cases, and capture terminal/output screenshots.
-

6. PROGRAM

File: *Dynamic.html*

Dynamic.html - Part 1 (Lines 1-45)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Weather DashBoard</title>
</head>
<body id="body">
  <form id="weatherForm">
    <input type="text" id="city" placeholder="Enter City">
    <button type="submit">Search</button>
    <button type="button" id="getLocation">Get Current Location</button>
    <button type="button" id="getTemp">Get Current temprature</button>
  </form>
  <h2 id="SelectedCity"></h2>
  <h2 id="temperature"></h2>
</body>
<script>
  document.getElementById("weatherForm").addEventListener("submit", function(e) {
    e.preventDefault(); // Prevent page refresh

    const city = document.getElementById("city").value;
    document.getElementById("SelectedCity").innerText = city;
    // getWeather(city);
  });

  document.getElementById("getLocation").addEventListener("click", function() {
    if (navigator.geolocation) {
      navigator.geolocation.getCurrentPosition(async function(position) {
        const lat = position.coords.latitude;
        const lon = position.coords.longitude;
        // getWeatherByCoordinates(lat, lon);
        try{
          const response = await fetch(
            `https://nominatim.openstreetmap.org/reverse?format=json&lat=${lat}&lon=${lon}`
          );
          const data = await response.json();
          const name = document.getElementById("SelectedCity");
          name.innerText = data.address.city || data.address.town || data.address.village || "Unknown
Location";
        }
        catch (error) {
          console.error("Error fetching location data:", error);
        }
      });
    }
  });
}
```

Dynamic.html - Part 2 (Lines 46-74)

```
});

document.getElementById("getTemp").addEventListener("click", function() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(async function(position) {
      const lat = position.coords.latitude;
      const lon = position.coords.longitude;
      try {
        const response = await fetch(
          `https://api.openweathermap.org/data/2.5/weather?lat=${lat}&lon=${lon}&appid=280668dd93f8ac56029ab0ecf13967b1`
        );
        const data = await response.json();
        // Process temperature data
        console.log("Data:", data)
        document.getElementById("temperature").innerText = `Temperature: ${data.main.temp - 273.15}
°C`;

        if (data.main.temp - 273.15 > 30) {
          document.getElementById("body").style.backgroundColor = "red";
        }
        else{
          document.getElementById("body").style.backgroundColor = "blue";
        }
      } catch (error) {
        console.error("Error fetching temperature data:", error);
      }
    });
  }
});
</script>
</html>
```

7. CODE EXPLANATION

JavaScript Code Segment	Explanation
<pre>addEventListener("submit", function(e) { e.preventDefault(); ... })</pre>	Attaches form submit handler and halts standard browser reload events, allowing JS text capture.
<pre>navigator.geolocation.getCurrentPosition(async function(pos) { ... })</pre>	Triggers GPS location capture via browser APIs, delivering latitude/longitude coords inside async handler.
<pre>await fetch(`https://nominatim.openstreetmap.org/...`)</pre>	Issues async HTTP GET request to OpenStreetMap reverse geocoding API to capture city details.
<pre>await fetch(`https://api.openweathermap.org/...`)</pre>	Issues async HTTP network query to OpenWeather API to retrieve coordinates' weather datasets.
<pre>data.main.temp - 273.15</pre>	Converts the returned OpenWeather temperature value from Kelvin to Celsius scale.
<pre>document.getElementById("body").style.backgroundColor = "red"</pre>	Mutates the background color of the HTML body tag dynamically to red when temperature exceeds 30°C.

8. TEST CASES AND EXECUTION DATA

The following test executions verify DOM updates and asynchronous Web API integration in Google Chrome:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	Form Submit Action	Type 'San Francisco' and click Search	Selected City placeholder renders value; page does not reload	Pass
TC-02	Reverse Geocoding	Click 'Get Current Location'	GPS coordinates sent to Nominatim; local city heading displays correctly	Pass
TC-03	Weather API Retrieval	Click 'Get Current temperature'	Queries OpenWeatherMap API and updates temperature placeholder	Pass
TC-04	Hot Temperature Style	Temperature result triggers value > 30°C	Body background style dynamically updates to red color	Pass
TC-05	Cool Temperature Style	Temperature result triggers value <= 30°C	Body background style dynamically updates to blue color	Pass

9. OUTPUT

Output: Rendered Weather Dashboard in Chrome Browser

Tagarapuvalasa

Temperature: 34.54000000000002 °C

10. RESULT

Thus, a dynamic webpage utilizing JavaScript (DOM event captures, innerText manipulations, Geolocation coordinates requests, and asynchronous network API fetches) was successfully developed and verified.

Submission Package Structure:

```
Experiment4-Week4/  
  ■■■ Docs/  
    ■ ■■■ Experiment 4.pdf  
  ■■■ Screenshots/  
    ■ ■■■ image.png  
  ■■■ src/  
    ■■■ Dynamic.html
```